

## CSE 421/521 PA1 Phase 3: Priority Donation & MLFQS

The objective of this assignment is to implement priority donation and MLFQS. See below for an explanation of priority donation; the phase 3 recitation video explains both objectives too:

- Your team will use the same PA1 repository from Phase 2. You will implement Phase 3 over your Phase 2 implementation.
- The goal of this phase is to:
  1. Implement priority donation and MLFQS
  2. Pass **all** PA1 tests
- One of the motivations behind priority donation is to fix the priority inversion problem. Suppose we have three threads  $L$ ,  $M$ , and  $H$  where the priorities of these threads are ordered like this:  $L < M < H$ . A thread with higher priority must run before threads with lower priorities as established in Phase 1.

If thread  $H$  is created after  $L$  acquires a lock that it needs, it will wait for thread  $L$  to release that lock, so  $H$  is now blocked by  $L$ . Then, thread  $M$  is created to run a function that requires lots of CPU time (say it spins in a loop for a while). Since  $M$  has higher priority than  $L$ ,  $L$  cannot be scheduled and release its lock while  $M$  is running. This means  $H$  cannot be scheduled because of  $M$  when the opposite should be true!

If  $H$  could somehow ‘donate’ its priority to  $L$  **temporarily while  $L$  is holding that lock**, then  $L$ ’s priority would be greater than  $M$ ’s, thus allowing  $L$  to be scheduled and release the lock that  $H$  needs. Once the lock is released,  $L$ ’s priority is reset back to its original priority, allowing  $H$  to be scheduled now. This is the essence of priority donation.

- For MLFQS, read Section 2.2.4 and all of Appendix B of the Pintos manual. The recitation video should also help you understand it better. You don’t need to worry about implementing fixed-point arithmetic; we have provided you an implementation found in `pintos/src/threads/fixedpoint.h`
- You need to find and edit relevant code in `pintos/src/threads`. We will be using Autolab to evaluate your work. You will submit your Phase 3 implementation the same way as Phase 2.
- If you plan to run this phase’s tests with `make check`, you’ll need to update the `Make.tests` file and add the remaining tests. You can find the updated `Make.tests` file on UBLearn - you may need to rename it to `Make.tests` after downloading it. The starter repository on GitHub also has these updates so you can copy the changes from there; [here’s a link to the commit](#).

You can find `Make.tests` in `pintos/src/tests/threads/`

- **Your team will need to submit PA1’s design documentation (designdoc) by the end of Phase 3 on UBLearn.** You need to answer the questions in `pintos/doc/threads.tmpl` in your submission.

This phase requires, by far, the most work in PA1 and can be very challenging compared to Phase 1 and 2. If you need help with this phase, come to office hours as often as you can.